LINUX FOUNDATIONS

A fact-checked theory reader

Course companion · English edition · 98-slide workshop

8	20	1
theory chapters	primary references	operating loop

LOCATE → PREDICT → ACT → OBSERVE → VERIFY

Technical review completed 18 July 2026
Ubuntu 24.04 classroom baseline · portable concepts marked explicitly

Course companion · English edition · verified 18 July 2026

This reader expands the theory behind the 98-slide Linux Foundations workshop. It is written for learners using an Ubuntu 24.04 classroom VM or Incus system container, while clearly marking behavior that differs by distribution, shell, or local policy. Commands are examples to inspect and discuss; make changes only in the course lab environment.

How to use this reader

Read Chapters 1–8 in order before the workshop, or use the command index and references afterward. Each chapter follows one operational loop:

- **Locate** the system, identity, object, or unit you are actually changing.
- **Predict** the result and the evidence that should prove it.
- **Act** with the narrowest reasonable command and privilege.
- **Observe** output, exit status, state, and logs.
- **Verify** behavior independently of the tool that made the change.

Scope note. "Linux" describes a kernel and a family of systems, not one identical user experience. This course uses GNU Bash, GNU coreutils, APT, OpenSSH, and systemd on Ubuntu. Rocky/RHEL systems commonly use DNF, may name the SSH unit `sshd.service`, and normally add SELinux policy as another access-control layer.

1. Linux, distributions, and execution environments

1.1 Kernel and user space

Linux is the kernel: the privileged resource manager that schedules processes, manages virtual memory, exposes filesystems and devices, and implements networking. Applications normally run in user space and request kernel services through system calls. A distribution combines a Linux kernel with user-space tools, libraries, an installer, package repositories, defaults, and a support policy [1][2].

A program file and a process are not the same thing. The file is executable content on storage. A process is a running instance with a process ID (PID), parent, credentials, memory mappings, environment, current directory, and open file descriptors. Two processes can execute the same program while having different users, environments, open files, and access rights.

Useful orientation commands:

```
cat /etc/os-release
uname -r
uname -m
ps -p 1 -o comm=
id
```

/etc/os-release identifies the operating-system user space. uname -r reports the running kernel release. In a container these can describe different providers: an Ubuntu user space can share the host's kernel.

1.2 What a distribution changes

Distribution families share kernel and Unix concepts, but package formats, repositories, security defaults, release cadence, service names, and file locations can differ. Debian and Ubuntu use .deb packages and APT. Fedora, RHEL, Rocky Linux, and AlmaLinux use RPM packages and DNF in current releases. Alpine commonly uses APK. Never translate a command by changing only the package-manager name; verify the package name, repository, service name, configuration path, and security policy.

Stable or long-term-support releases prioritize a maintained, predictable platform. Rolling releases continuously deliver newer package versions. Neither model is automatically "secure" or "insecure"; security depends on active maintenance, enabled updates, repository trust, configuration, and operational practice.

1.3 Virtual machines and containers

A conventional virtual machine runs a guest kernel on virtual hardware. An operating-system container shares the host kernel while giving the instance its own user space and namespaces. Incus can manage both system containers and virtual machines [3]. A system container can run systemd, users, packages, SSH, and logs, which makes it useful for this course.

VMs and containers are both isolation technologies, but "VM equals secure" is too broad. The actual boundary depends on hypervisor or kernel design, configuration, device exposure, privileges, updates, and workload. Closing a browser terminal ends the client connection; it does not inherently destroy the Incus instance.

1.4 A reliable orientation record

Before following external instructions, record:

```
cat /etc/os-release          # distribution and release
uname -r                     # running kernel
uname -m                     # architecture
command -v apt dnf apk       # available package front ends
ps -p 1 -o comm=             # process 1 / likely init manager
id                           # current identity and groups
sudo -l                      # permitted sudo commands, if authorized
```

Fact-check result. The course's kernel/user-space and VM/container distinctions are sound. The reader narrows the original "stronger VM boundary" wording because isolation strength is contextual, not an unconditional property.

2. The Linux filesystem and safe file operations

2.1 One namespace, many filesystems

Linux presents a single pathname hierarchy rooted at /. A mount attaches another filesystem at a directory in that hierarchy. Therefore /home may reside on another disk without changing a user's pathname. findmnt describes mounts, df reports space by filesystem, and du estimates space attributable to directory trees.

The Filesystem Hierarchy Standard (FHS) defines common purposes, but distributions and applications may extend or depart from it [4]. Useful starting points include:

- /etc: host-specific configuration.
- /usr: shareable, mostly read-only distribution-supplied programs and data.
- /var: variable state such as logs, queues, caches, and databases.
- /home: ordinary users' home directories.
- /run: volatile runtime data since boot.
- /tmp: temporary files; retention and cleanup are policy-dependent.
- /proc: a virtual view of processes and kernel information.
- /sys: a virtual view of devices, drivers, and kernel objects.

/proc and /sys are not ordinary backup data. Some entries are generated on read, and some writable entries alter the running kernel. Read them for inspection; write only with documented intent.

2.2 How paths resolve

An absolute path starts with / and begins at the process's root directory. A relative path begins at the current working directory. During pathname resolution, Linux traverses directory components and checks search (execute) permission on directories [5].

```
pwd
realpath ../target
stat ../target
namei -l ../target
```

. denotes the current directory and .. its parent. ~ is shell syntax expanded to a home directory; it is not a literal filesystem component. ./script.sh explicitly names a file in the current directory, avoiding reliance on whether . appears in PATH.

Names beginning with . are omitted by many default directory listings. That is a visibility convention, not access control. Permissions still determine who can traverse a directory or read a file.

2.3 Files, inodes, and links

A directory maps names to filesystem objects. ls -l begins each entry with a type indicator: - regular file, d directory, l symbolic link, c character device, b block device, p named pipe, or s socket. stat reports metadata; file inspects content and other clues to classify a path.

A hard link is another directory entry for the same inode. It normally cannot cross filesystem boundaries, and ordinary users cannot hard-link directories. Removing one name does not remove the data while another hard link remains and a process may also keep an unlinked file open. A symbolic link stores a pathname, can cross filesystems, can refer to directories, and can dangle when its target cannot be resolved [6].

2.4 Shell expansion precedes the command

In Bash, parsing and expansion transform the typed command into an argument list. Expansions include parameter expansion, command substitution, word splitting, filename expansion (globbing), and quote removal in a defined order [7]. The called program usually receives expanded names, not *.log itself.

```
printf '<%s>\n' -- *.log
printf '<%s>\n' -- "$HOME/course notes"
```

Single quotes preserve characters literally. Double quotes still permit parameter and command substitution but prevent word splitting and pathname expansion of the result. Quote data by default.

2.5 Copy, move, and remove safely

GNU coreutils documents cp, mv, and rm; options and edge behavior should be checked locally with --help or man [8]. A safe change sequence is:

1. Confirm context with pwd, id, and realpath.
2. Enumerate the exact targets with printf, find -print, or a non-destructive listing.
3. State the expected target count and postcondition.
4. Run the smallest scoped change.
5. Verify with stat, cmp, find, or an application-level test.

Interactive flags can reduce accidents but do not replace target verification. Never construct a destructive command around an unchecked, possibly empty variable.

Fact-check result. The filesystem theory is correct after two clarifications: ~ is a shell expansion, and “file equals inode” is an oversimplification across all filesystem types. The reader uses “filesystem object” where inode behavior is not guaranteed.

3. Terminal, shell, streams, and text tools

3.1 Separate the layers

A terminal provides an interactive input/display channel. A shell such as Bash parses language, performs expansions and redirections, launches commands, and reports status. A command is a program or shell construct plus the final arguments produced after shell processing. The browser's ttyd front end and macOS Terminal can both connect to a Bash session; the interface differs while the remote shell behavior remains the same.

3.2 Standard streams and redirection

Processes conventionally begin with standard input (file descriptor 0), standard output (1), and standard error (2). Redirection is processed by the shell [7].

```
command >result.txt 2>error.txt
command >combined.txt 2>&1
producer | consumer
```

`>` truncates or creates a file for output; `>>` appends. The order of redirections matters because `2>&1` duplicates the destination currently used by standard output. A pipe connects one command's standard output to another command's standard input; it does not automatically include standard error.

3.3 Exit status and control flow

Bash treats status 0 as success and non-zero as failure for conditional execution. Status 126 conventionally means a command was found but could not execute, 127 means command not found, and a command terminated by signal `N` is represented by `128+N` in Bash [9]. Other non-zero meanings belong to the command; for `grep`, status 1 means no selected lines, while status 2 indicates an error.

```
some-command
printf 'status=%s\n' "$?"

prepare && apply
primary || fallback
```

Inspect `$?` immediately because the next command replaces it. `A && B` runs `B` only when `A` succeeds. `A || B` runs `B` only when `A` fails.

By default, a Bash pipeline reports the status of its final command. With `set -o pipefail`, it reports failure when any stage fails, using Bash's documented rightmost-nonzero rule [10]. This is why a successful `head` or `sort` can otherwise conceal an earlier producer failure.

3.4 Evidence-oriented pipelines

Build pipelines from a precise question:

6. Select the smallest relevant source.
7. Filter records by explicit criteria.
8. Transform or extract fields.
9. Sort, count, or aggregate only after inspecting samples.
10. Retain diagnostics until you know they are irrelevant.

```
awk '{print $9}' access.log | sort | uniq -c | sort -nr
```

This example assumes field 9 really is the desired value; inspect the input format first. `2>/dev/null` removes evidence and should be used only when the omitted failures are expected and understood.

3.5 Terminal editors

For Nano, the essential model is: open a named file, edit text, write the buffer, and exit. For Vim, distinguish Normal mode from Insert mode; `i` enters Insert mode, `Esc` returns to Normal mode, `:w` writes, and `:q` quits. A successful save is not proof that the application parsed or applied the configuration. Follow editing with a syntax check, reload/restart where appropriate, and behavioral verification.

Fact-check result. The course's stream, quoting, and status model matches the Bash manual. The reader makes clear that pipeline status is shell-specific and that `pipefail` is an option, not the default.

4. Packages and software provenance

4.1 What a package manager does

A package manager relates package names and versions to trusted repositories, resolves dependencies, verifies repository metadata and package integrity according to its trust model, installs files, executes package lifecycle scripts, and records package state. It does not make every third-party repository safe.

On Ubuntu 24.04, APT reads repository definitions primarily from /etc/apt/sources.list.d/, with Ubuntu's deb822-format configuration normally in ubuntu.sources [11]. Older Ubuntu releases commonly use /etc/apt/sources.list.

```
sudo apt update
apt policy tree
sudo apt install tree
dpkg -s tree
command -v tree
```

apt update refreshes the local package index; it does not upgrade installed packages. apt install resolves and installs the requested package and dependencies. dpkg -s inspects the local package database. command -v verifies how the shell would resolve a command name. Test the program itself for behavioral proof.

4.2 Interactive versus scripted APT

Ubuntu documents apt as an interactive command-line interface and recommends apt-get for scripts because apt output and behavior are less stable as a scripting interface [11]. Automation also needs an explicit policy for prompts, service restarts, failure handling, and locks; simply adding -y is not a complete automation design.

4.3 Removal, configuration, and logs

apt remove normally removes the package while retaining certain configuration. apt purge also requests removal of package configuration tracked by the package system. Neither promise should be read as "erase every file the application ever created"; user data, runtime data, and administrator-created files may remain.

Ubuntu records package actions in /var/log/dpkg.log; APT history is commonly available in /var/log/apt/history.log [11]. Journal entries may also show service starts or failures triggered by package installation.

4.4 Cross-distribution translation

DNF and APT solve similar lifecycle problems but are not syntactic aliases. Repositories, package names, modularity, signatures, transaction history, and default policies differ. On RHEL-family systems, SELinux can deny an operation even when traditional owner/group/mode bits appear permissive. Treat the platform's security model as part of the package and service investigation.

Fact-check result. The package workflow is correct for Ubuntu 24.04. The reader corrects the common overstatement that purge removes "all configuration," and marks APT paths and log locations as Ubuntu/Debian-family details.

5. Users, groups, ownership, and permissions

5.1 Identity comes from more than local files

A process carries user and group credentials used for access checks. /etc/passwd and /etc/group describe local accounts and groups, but the Name Service Switch can resolve identities from LDAP, Active Directory integrations, or other providers. Use getent passwd NAME and getent group NAME when asking whether the system can resolve an identity [12].

```
id
id alice
getent passwd alice
getent group developers
```

Do not infer “human user” from a fixed UID boundary across every distribution. UID allocation ranges are configured policy; Ubuntu commonly begins regular users at 1000, while system accounts occupy lower ranges [12].

5.2 How mode bits are selected

Traditional discretionary access control has owner, group, and other classes, each with read (r), write (w), and execute/search (x) bits. During an access check, Linux selects one class: owner when the process’s effective UID matches the file owner; otherwise group when an effective or supplementary group matches; otherwise other [5]. It does not combine the most generous bits from all three classes.

For a regular file:

- read permits reading file content;
- write permits modifying content, subject to other controls;
- execute permits execution when the format, mount, interpreter, and security policy also allow it.

For a directory:

- read permits listing names;
- write permits creating/removing/renaming entries, normally together with search permission;
- execute means search/traversal through the directory.

Deleting a file is principally an operation on its parent directory. A read-only file can be deleted by a user who has the required directory permissions, unless sticky-bit, immutable-attribute, mount, ACL, MAC, or other controls prevent it.

5.3 Symbolic and numeric modes

```
chmod u=rw,g=r,o= report.txt
chmod 640 report.txt
chmod g+w shared
```

Numeric modes add read=4, write=2, execute=1 within each class. 640 means owner read/write, group read, and no other permissions. Symbolic modes often communicate intent more clearly during review. Avoid chmod 777: it grants every class read, write, and execute/search and usually destroys the boundary you intended to create.

chown USER:GROUP PATH changes ownership; chgrp GROUP PATH changes only group ownership. Recursive changes require special care around symlinks, mount points, generated data, and mixed ownership. GNU coreutils documents traversal options and their security implications [13].

5.4 Umask and default permissions

The umask removes permission bits from the mode requested by a creating program. It does not add permissions and it is not itself a final file mode. Typical programs request 0666 for new regular files and 0777 for new directories, then the kernel applies the process umask. Defaults can also be changed by ACLs, application behavior, mount options, and security policy.

5.5 Privilege and sudo

UID 0 is traditionally privileged, while modern Linux also splits some privilege into capabilities. sudo applies policy to run a command as another user, commonly root. Ubuntu's initial installer-created account is normally authorized through the sudo group [12]. This is an Ubuntu policy, not a universal Linux rule.

Prefer one explicit privileged command over a long root shell. Before adding sudo to a failing command, determine whether the operation should require privilege. A pathname error, wrong service name, missing package, or syntax error will not be fixed by broader authority.

5.6 Other access-control layers

Mode bits are not the whole decision. POSIX ACLs, SELinux or AppArmor, mount flags such as noexec, immutable attributes, read-only filesystems, capabilities, namespaces, and application policy can permit or deny behavior. Inspect the layer suggested by the evidence instead of setting broader mode bits.

Fact-check result. The core permission model is accurate. The reader adds the crucial "one matching class" rule, directory deletion semantics, NSS identity sources, and non-mode access controls.

6. SSH trust, keys, and safe client configuration

6.1 SSH solves two authentication questions

SSH establishes an encrypted connection, authenticates the server to the client using host keys, and authenticates the client/user to the server using a configured method such as a public key [14]. These are separate trust decisions.

On first contact, a client may ask whether to trust a server host-key fingerprint. Verify that fingerprint through an independent trusted channel before accepting it. The accepted key is stored in known_hosts according to client configuration. A later mismatch can indicate legitimate reinstallation or address reuse, but it can also indicate interception; investigate rather than deleting the warning reflexively.

6.2 User key pairs

For a new general-purpose key, Ubuntu and OpenSSH documentation support Ed25 519 where compatible [14] [15].

```
ssh-keygen -t ed25 519 -a 64 -f ~/.ssh/course_ed25 519
ssh-keygen -lf ~/.ssh/course_ed25 519.pub
```

The private key remains on the client and should be protected by file permissions and, normally, a passphrase. The public key can be installed on the target account, commonly in ~/.ssh/authorized_keys. A passphrase protects the private key at rest; an ssh-agent can cache authorization to use it.

ssh-copy-id user@host is a convenience tool present on many Unix-like clients, but it is not guaranteed on stock Windows installations. A portable course should also show how to copy the public-key line through an existing authenticated session.

6.3 Permissions and path traversal

OpenSSH can reject insecure key files or ignore an authorized_keys path that is writable by other users, depending on StrictModes and server configuration [16]. Inspect every component:

```
namei -l ~/.ssh/authorized_keys
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

These are conservative common modes, not magic numbers that override ownership or higher-level policy. The file must belong to the intended account and the complete path must be traversable as required.

6.4 Client configuration

~/.ssh/config reduces repeated arguments and can bind a friendly alias to a host, user, port, and identity:

```
Host course-server
  HostName 192.0.2.20
  User learner
  IdentityFile ~/.ssh/course_ed25 519
  IdentitiesOnly yes
  ServerAliveInterval 30
```

Inspect the evaluated configuration with:

```
ssh -G course-server | less
ssh -vv course-server
```

ssh -G prints the resulting client configuration without connecting. ssh -vv provides diagnostic detail during connection setup; review logs before sharing because hostnames, usernames, addresses, and key paths may be sensitive.

StrictHostKeyChecking no weakens host authentication and should not be the classroom default. If automation must pre-provision host keys, obtain and validate them through a trustworthy channel; ssh-keyscan alone retrieves keys but does not authenticate their origin [15].

6.5 Server-side changes without lockout

On Ubuntu the server package is openssh-server, and the service is normally ssh.service; other distributions often use sshd.service [14]. Before restarting a remotely administered server:

```
sudo sshd -t
sudo systemctl reload ssh.service
```

Validate configuration first, keep the existing session open, open a second test connection, and preserve a recovery path. Whether reload is sufficient depends on the specific setting and service implementation; follow platform documentation.

Fact-check result. The key setup and client model are sound. The reader removes any implication that accepting a first-seen key proves identity, marks ssh-copy-id as non-universal, and warns that ssh-keyscan is collection—not verification.

7. Logs and journalctl

7.1 Logs are event evidence, not complete truth

Logs are records produced by components under particular configuration. Missing output can mean no event, wrong time range, wrong unit, insufficient permission, rate limiting, rotation, volatile storage, or logging elsewhere. Begin with a bounded question: which host, boot, unit, time window, priority, and identity?

Traditional text logs commonly live under /var/log, but filenames differ by distribution and service. On systemd systems, systemd-journald collects structured journal entries, and journalctl queries entries accessible to the caller [17].

7.2 High-value journal queries

```
journalctl -u ssh.service -b --since '-15 min' --no-pager
journalctl -p warning..alert -b --no-pager
journalctl -k -b -1 --no-pager
journalctl -u course-web.service -f
```

- -u filters by unit and related systemd messages.
- -b limits to a boot; -b -1 requests the preceding boot when retained.
- --since and --until bound time.
- -p filters priorities.
- -k selects kernel messages.
- -f follows newly appended entries.

The journal contains structured fields such as _SYSTEMD_UNIT, _PID, _UID, _COMM, and PRIORITY. Field matches can make a query more precise than text search [17].

7.3 Persistence, rotation, and access

Journal persistence is configuration-dependent. With volatile storage, entries reside under /run/log/journal and disappear at reboot. Persistent journals normally live under /var/log/journal. The Storage= setting and directory availability influence the selected mode [18]. Do not promise that journalctl -b -1 will work unless previous-boot logs were retained.

Access is also policy-dependent. Root and users in appropriate groups can normally see more system entries than ordinary users. A learner who sees fewer entries may be encountering access control, not absence of events.

7.4 A diagnostic reading method

11. Reproduce or identify the exact symptom and time.
12. Query the smallest relevant unit, boot, and time window.
13. Read chronologically and find the earliest actionable error.
14. Widen one dimension at a time: dependency, neighboring unit, kernel, or earlier time.
15. Save the bounded query and verify after one repair.

Follow mode is useful while reproducing behavior, but it does not show events that happened before follow began unless they are included in initial output. Capture a historical baseline first.

Fact-check result. The journalctl examples match systemd documentation. The reader explicitly makes retention and access conditional, correcting the common assumption that previous-boot logs always exist.

8. systemd services and evidence-based troubleshooting

8.1 Units and load paths

systemd manages units representing services, sockets, timers, mounts, targets, and other resources. Unit files are loaded from a precedence-ordered search path that can include vendor, administrator, runtime, generator, and user locations. Exact vendor paths vary by distribution and build [19]. Therefore, do not teach `/usr/lib/systemd/system` as universal.

Use the manager's effective view:

```
systemctl cat ssh.service
systemctl show ssh.service -p FragmentPath -p DropInPaths
systemd-analyze unit-paths
```

Durable administrator overrides normally belong under `/etc/systemd/system`, often as drop-ins created with `systemctl edit`. Runtime changes under `/run` disappear at reboot. Avoid editing package-supplied vendor files because upgrades can replace them.

8.2 Runtime state versus boot policy

These are independent questions:

- `systemctl start NAME`: request activation now.
- `systemctl stop NAME`: request deactivation now.
- `systemctl restart NAME`: stop and start again.
- `systemctl reload NAME`: ask a running service to reload, only when supported.
- `systemctl enable NAME`: establish the unit's configured enablement links for future activation; it does not prove the service is healthy now.
- `systemctl enable --now NAME`: request enablement and start in one command.
- `systemctl daemon-reload`: make the manager reread unit definitions; it does not by itself restart services.

`active`, `enabled`, and "application works" are three different facts [20].

8.3 Dependencies and ordering

Activation relationships and ordering relationships are separate. `Wants=` is a weaker activation dependency than `Requires=`. `After=` and `Before=` express order when both units participate; they do not pull a missing unit into the transaction by themselves [19]. `After=network.target` also does not guarantee that an application-specific network endpoint is reachable.

```
systemctl list-dependencies course-web.service
systemctl show course-web.service -p Wants -p Requires -p After
```

8.4 State, events, definition, behavior

Use four evidence planes:

16. **State:** `systemctl status`, `is-active`, and `show` summarize the manager's current view.
17. **Events:** `journalctl -u NAME -b` gives the time-ordered sequence.
18. **Definition:** `systemctl cat` and selected `show` properties expose effective configuration.
19. **Behavior:** a real client test proves whether the service delivers its intended function.

```
systemctl status course-web.service --no-pager
journalctl -u course-web.service -b --since '-10 min' --no-pager
systemctl cat course-web.service
sudo systemctl restart course-web.service
systemctl is-active course-web.service
curl --fail http://127.0.0.1:8088/
```

An active process may still return errors, listen on the wrong interface, serve stale data, or fail authorization. Conversely, an oneshot unit can be successfully inactive after completing. Interpret state according to unit type, then test behavior.

8.5 The complete troubleshooting loop

20. State the user-visible symptom without explaining it yet.
21. Identify host, distribution, identity, unit, and time.
22. Capture current state and the earliest relevant journal event.
23. Inspect the effective unit and named dependencies.
24. Form one hypothesis that explains the evidence.
25. Change only the evidenced cause.
26. Re-run state, log, and independent client checks.
27. Record what changed and how rollback would work.

Fact-check result. The systemd operating model is accurate. The main correction is portability of the unit load path. The reader also clarifies that reload, successful active, and successful enable each prove different things.

9. Fact-check summary and corrections to retain

The underlying course theory is technically strong. The review did not find a need to change the course's operational model, but these qualifications are important:

- "Linux" is not one distribution; package, service, and policy details must be labeled.
- VM isolation is not unconditionally stronger; configuration and threat model matter.
- ~ is expanded by the shell, not resolved as a literal filesystem component.
- Bash pipeline status comes from the final command unless pipefail is enabled.
- /etc/passwd is not necessarily the complete identity source; use NSS-aware tools.
- Deleting a file depends chiefly on parent-directory permissions, plus other controls.
- apt purge does not guarantee erasure of all application or user-created data.
- First-contact SSH key acceptance does not independently verify server identity.
- ssh-keyscan collects a key but does not authenticate it.
- Previous-boot journal data exists only if it was retained.
- systemd vendor-unit paths vary; inspect the effective unit instead of assuming a path.
- enabled, active, and behaviorally healthy are independent facts.

10. Command quick reference

```
# Orientation
cat /etc/os-release
uname -r; uname -m
id; pwd

# Files and paths
realpath PATH
namei -l PATH
stat PATH
findmnt

# Packages (Ubuntu classroom)
sudo apt update
apt policy PACKAGE
sudo apt install PACKAGE
dpkg -s PACKAGE

# Users and permissions
getent passwd USER
id USER
chmod 640 FILE
chown USER:GROUP FILE

# SSH
ssh-keygen -t ed25519 -a 64
ssh-keygen -lf KEY.pub
ssh -G HOST
ssh -vv HOST

# Logs
journalctl -u UNIT -b --since '-15 min' --no-pager
journalctl -p warning..alert -b --no-pager

# Services
systemctl status UNIT --no-pager
systemctl cat UNIT
systemctl show UNIT -p FragmentPath -p DropInPaths
systemctl is-active UNIT
systemctl is-enabled UNIT
```

References

[1] Linux kernel documentation, *The Linux Kernel documentation*: https://docs.kernel.org/

[2] Linux Foundation, *Filesystem Hierarchy Standard 3.0*: https://refspecs.linuxfoundation.org/FHS_3.0/fhs-3.0.html

[3] Incus documentation, *About instances*: https://linuxcontainers.org/incus/docs/main/explanation/instances/

[4] Linux Foundation, *Filesystem Hierarchy Standard 3.0*: https://refspecs.linuxfoundation.org/FHS_3.0/fhs-3.0.html

[5] Linux man-pages project, *path_resolution(7)*: https://man7.org/linux/man-pages/man7/path_resolution.7.html

[6] Linux man-pages project, *symlink(7)* and GNU coreutils, *ln*: https://man7.org/linux/man-pages/man7/symlink.7.html and https://www.gnu.org/software/coreutils/manual/coreutils.html

[7] GNU Bash, *Shell Expansions and Redirections*: https://www.gnu.org/software/bash/manual/bash.html#Shell-Expansions

[8] GNU coreutils, *Basic operations*: https://www.gnu.org/software/coreutils/manual/html_node/Basic-operations.html

[9] GNU Bash, *Exit Status*: https://www.gnu.org/software/bash/manual/html_node/Exit-Status.html

[10] GNU Bash, *Pipelines*: https://www.gnu.org/software/bash/manual/html_node/Pipelines.html

[11] Canonical, *Install and manage packages*: https://documentation.ubuntu.com/server/how-to/software/package-management/

[12] Canonical, *User management*: https://documentation.ubuntu.com/server/how-to/security/user-management/

[13] GNU coreutils, *Changing file attributes*: https://www.gnu.org/software/coreutils/manual/coreutils.html#Changing-file-attributes

[14] Canonical, *OpenSSH server*: https://documentation.ubuntu.com/server/how-to/security/openssh-server/

[15] OpenBSD manual pages, *ssh-keygen(1)* and *ssh_config(5)*: https://man.openbsd.org/ssh-keygen.1 and https://man.openbsd.org/ssh_config.5

[16] OpenBSD manual pages, *sshd(8)* and *sshd_config(5)*: https://man.openbsd.org/sshd.8 and https://man.openbsd.org/sshd_config.5

[17] systemd, *journalctl(1)*: https://www.freedesktop.org/software/systemd/man/latest/journalctl.html

[18] systemd, *journald.conf(5)*: https://www.freedesktop.org/software/systemd/man/latest/journald.conf.html

[19] systemd, *systemd.unit(5)*: https://www.freedesktop.org/software/systemd/man/latest/systemd.unit.html

[20] systemd, *systemctl(1)*: https://www.freedesktop.org/software/systemd/man/latest/systemctl.html

Review provenance

Technical claims were checked against primary upstream manuals and current Ubuntu Server documentation on 18 July 2026. The course's open-source inspiration remains attributed separately in resources/SOURCES.md. URLs are intentionally written in full so they remain useful in printed and offline copies.